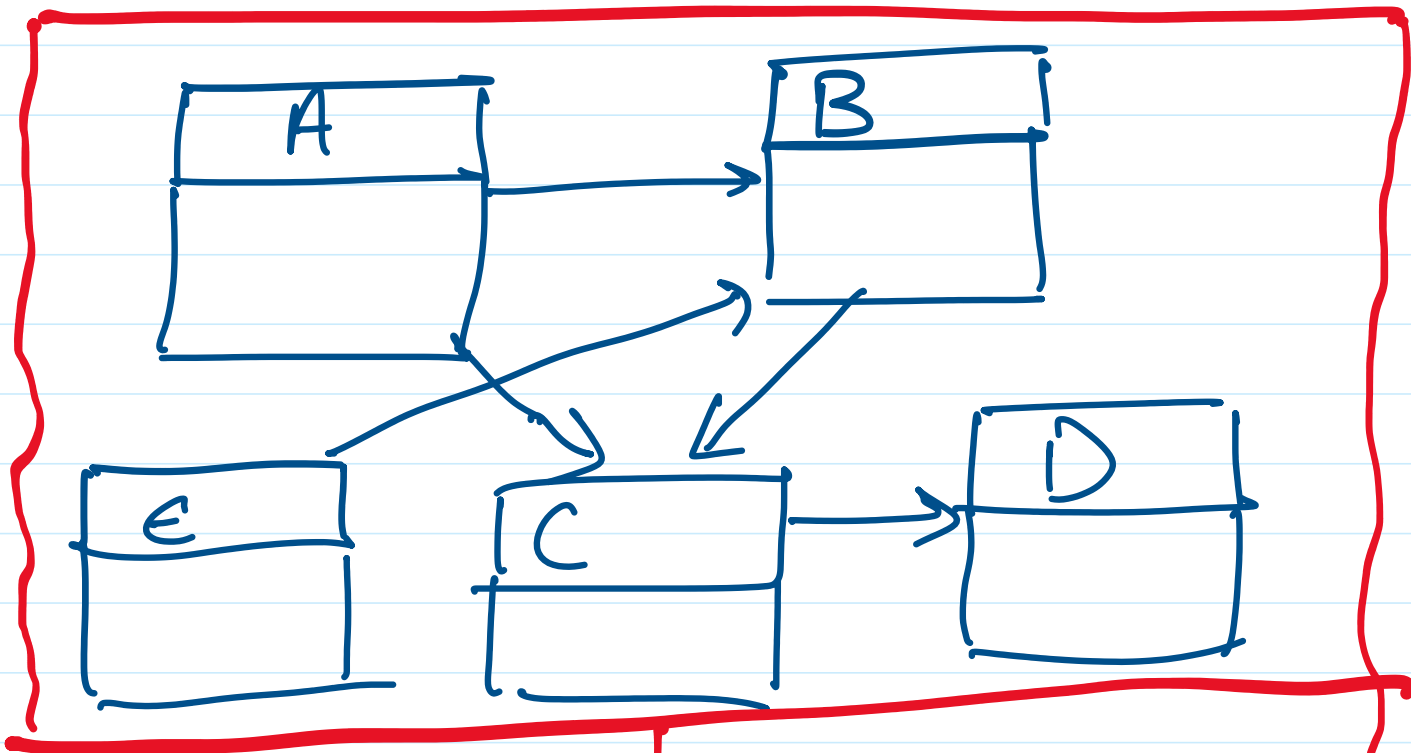


Facade Design Pattern

Suppose we have lot of Classes and they are complexly dependent on each other.

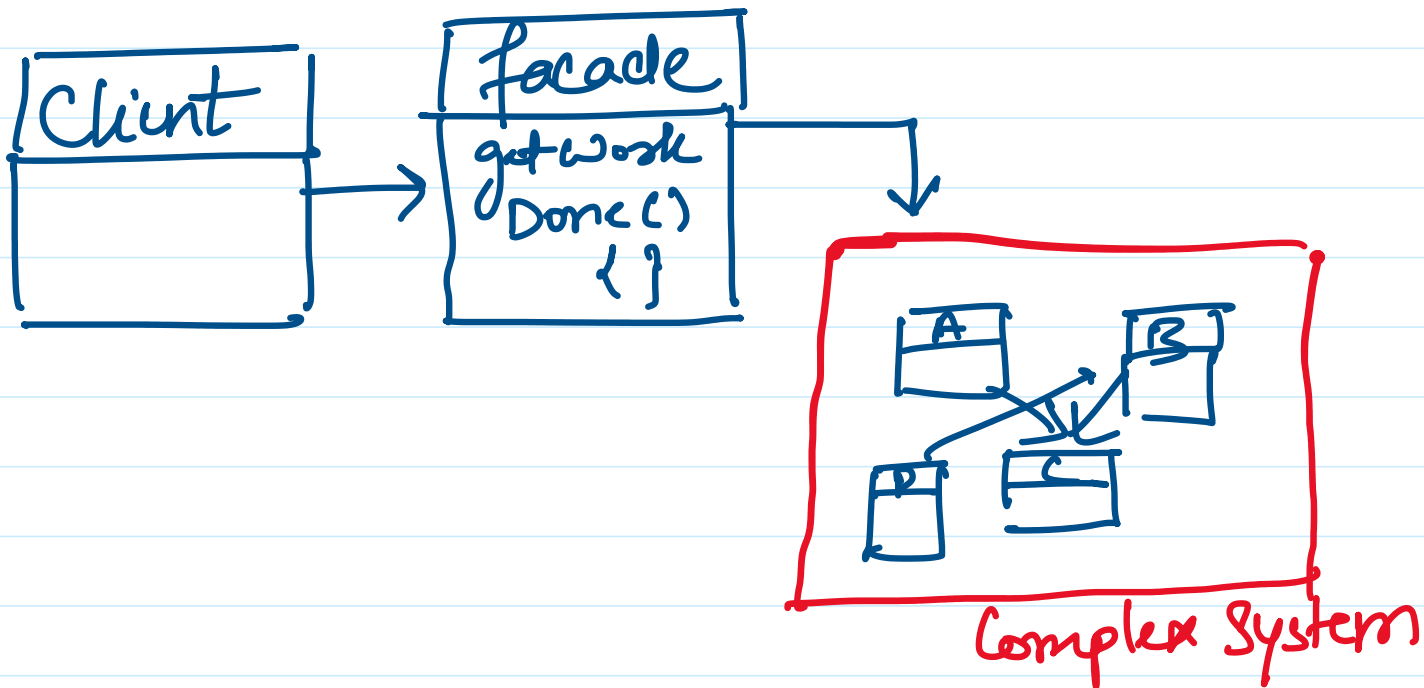


Together they are ~~are~~ doing something

Now there are some clients who want to work with this system but don't want to interact one by one with every class and perform its work

So we introduce new Class called

So we introduce new class called facade which talks with complex system and give result to Client

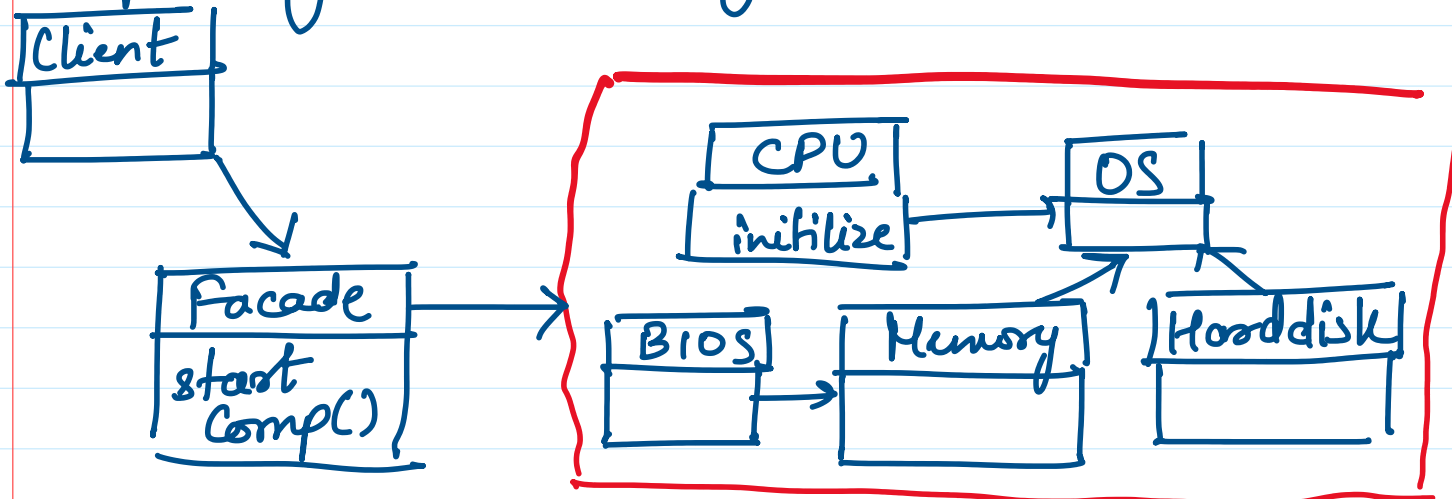


Client will simply call `getWorkDone()` and it doesn't know how facade did it.

- So, it decouple Client with complex sub-system, so in future if you want to introduce something new in complex sub-system then you can.
- It also satisfy - "Principal of least Knowledge"

Example:-

when you press your computer Button it gets 'on', you don't know what happening in background

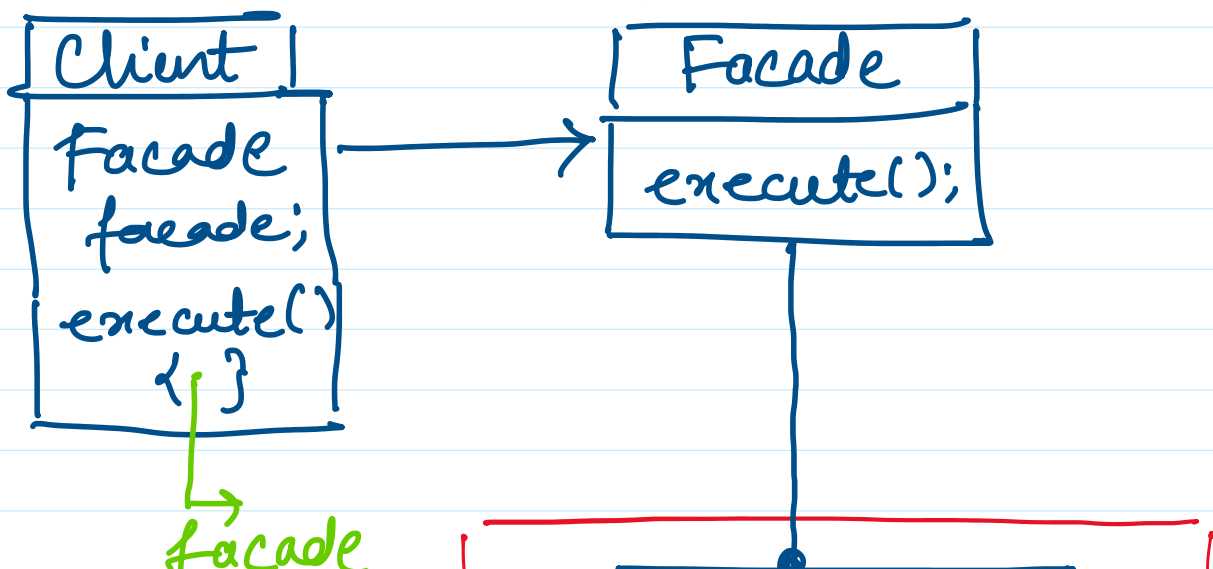


Standard Diagram of Above

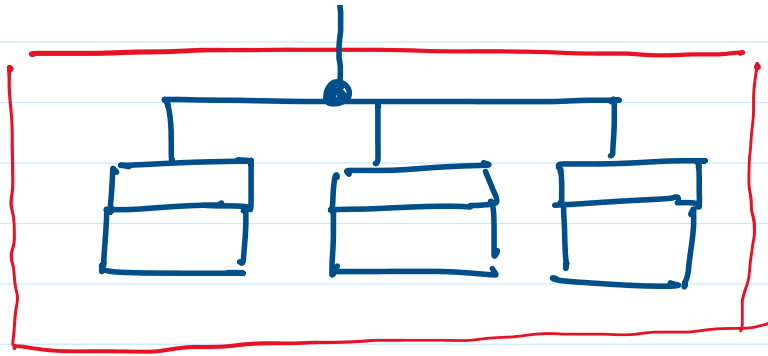
→ see Notes...

Example UML .jpeg

Standard UML Diagram

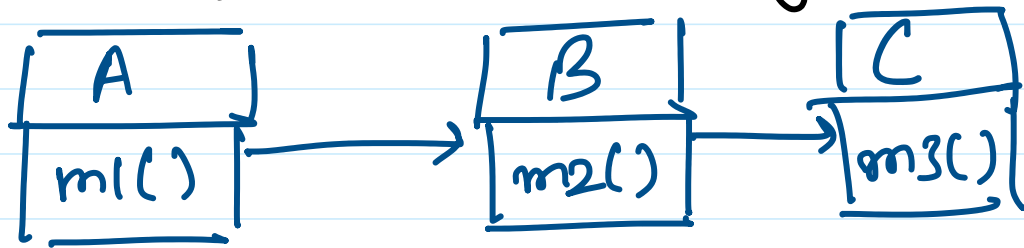


↳ facade
→ execute



Code → see Notes...

Principal of least knowledge



Talk only to immediate friend

A should only call B's method not C's methods directly.

why?

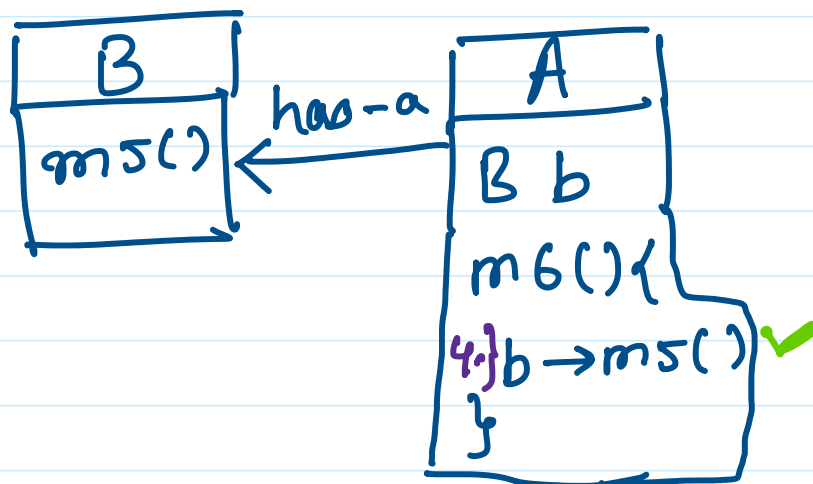
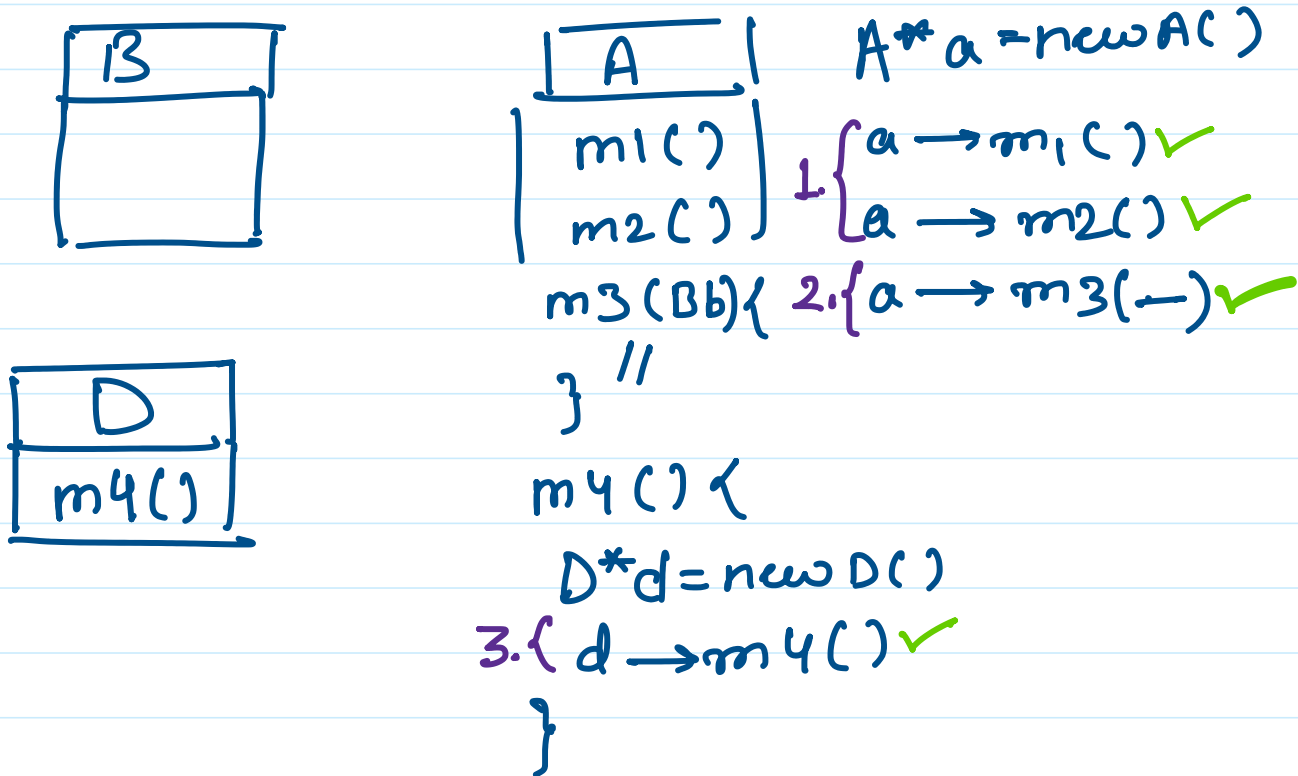
More the Interaction b/w Classes reduces more our application will be loosely-coupled

Principal of least knowledge (RULES):

Take any object, now from any method in that object, principal tells you to invoke only methods that belong to:

1. The object itself
2. The object passed in as a parameter to the method

3. Any object that method Creates.
4. Any object with (HAS-A) relationship

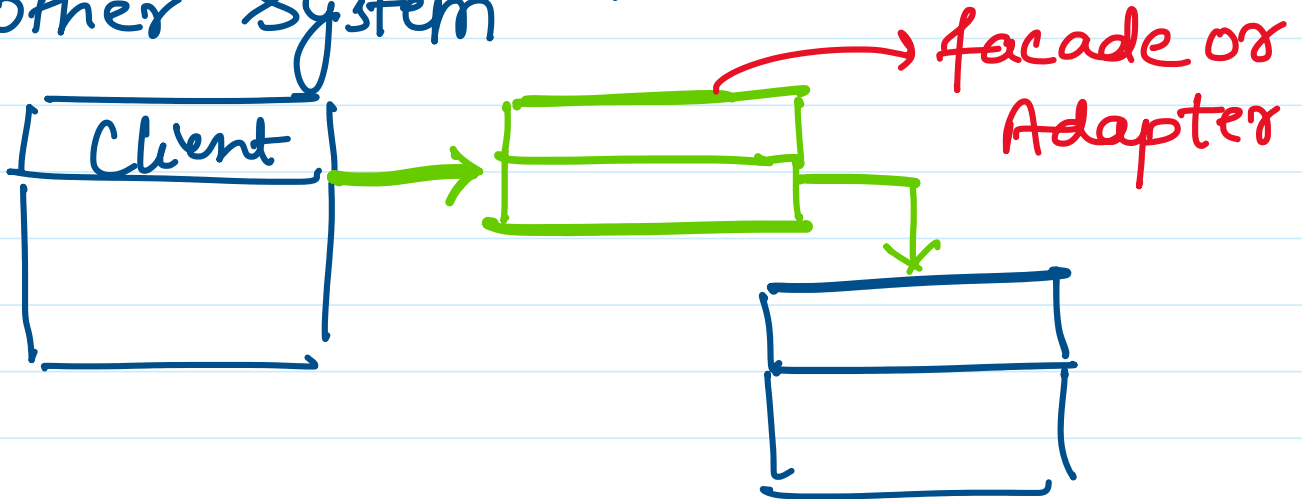


Definition

Facade pattern provides a simplified, unified interface to a set of complex subsystem. It hides the complexity of the system and exposes only what is necessary

Facade v/s Adapter Pattern

Both are doing one thing Introducing a class in b/w client & other system



Difference is b/w their Intent :-

Facade → Hides the Complexity of Sub-system

Adapter → To make the interaction

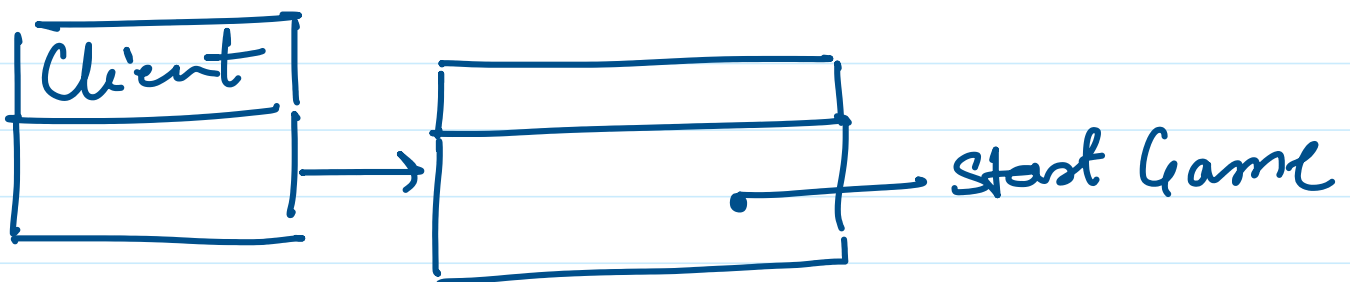
Adapter → To make the interaction
b/w Completely diff. interfaces

Use Cases:-

- Gaming Engine

Ex - Unity

You just have 'start Button' to start the game but in background lot of things are happening in Background



Complex sub-system

↳ Game Asset load
↳ Memory Management
↳ Physics Engine

- Payment Gateway

You just click on make payment
and in background -

- Checks balance
- Pin check
- fraud Detection
- sending Notification

Complex
sub-system